La Sénia
Institut d'Educació Secundària
Paiporta

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

Programación Orientada a Objetos (POO)

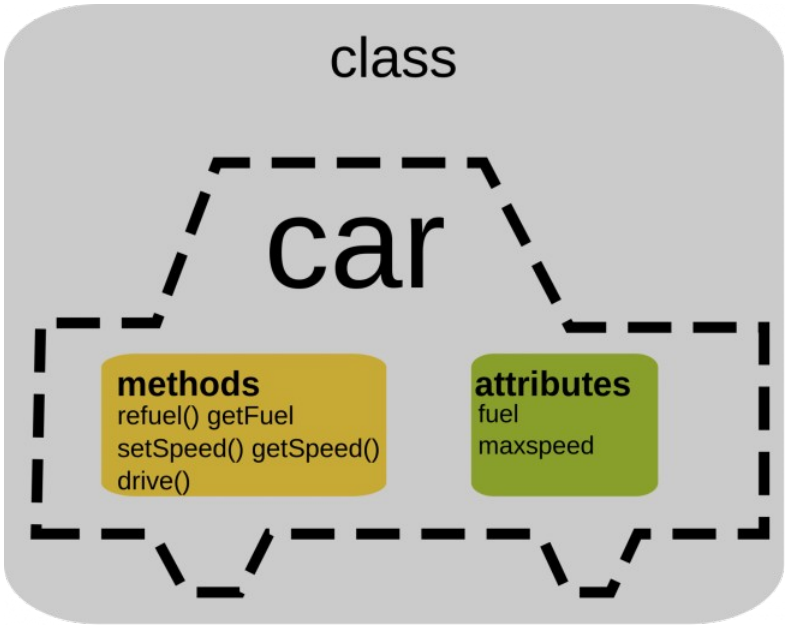
Curso 2025 - 2026

Programación Orientada a Objetos

Modelo de programación con elementos a programar llamados **objetos.**

- Propiedades (**atributos**)
- Acciones (**comportamiento**).

Ejemplo:
Un **coche** es un **objeto**.
Los **atributos** definen su estado.
Los **métodos** definen cómo se comporta



1. Introducción a la POO

Conceptos:

- Abstracción
- Clases y Objetos
- Atributos y Métodos
- Encapsulamiento
- Constructores
- Getters y Setters

Abstracción

Capacidad de definir las características de los objetos que nos rodean.

Ejemplo. El concepto de **mesa** puede ser distinto para cada persona (mesa de comedor, pupitre, mesa de despacho, mesa de cocina, mesita de noche, mesa reuniones)

Abstracción:

"Una mesa se intuye como una superficie plana, sujeta por unas patas y que sirve para realizar tareas como escribir, comer, etc. Puede ser de varios materiales y colores."

2. Abstracción

Propiedades:

- **estado**
- **color**
- **material**
- **número de patas**
- **tamaño**
- **superficie**

Acciones:

- **comerEnMesa()**
- **prepararMesa()**
- **quitarMesa()**
- **estudiarEnMesa()**
- **trabajarEnMesa()**
- **subirseAMesa()**

3. Clases y Objetos

Clase

Es una **plantilla** que define **cómo son los objetos** que pertenecen a ella. Se compone de:

- **Información**: Los **atributos** o propiedades
- **Comportamiento**: Los **métodos** (funciones y/o procedimientos)

Objeto

Es una **instancia de una clase**. (Crear un objeto de una clase)

Las propiedades toman valores y los métodos se pueden ejecutar.

Dos objetos de una misma clase tienen los mismos **atributos** y **métodos**.

3. Clases y Objetos

Clase

Concepto **abstracto** que define unas cualidades. Un *coche* cualquiera

Instancia

Objeto concreto de una clase. Mi *coche*

Atributos

Características que tienen todos los objetos de una clase. Matrícula, marca, modelo, color, numero de plazas,….

Métodos

Acciones asociadas a los objetos. Arranca, para, acelera, aparca,...

Ejemplos en java

```java
/* Coche.java */
public class Coche {
    //Atributos
    String matricula;
    String marca;
    String modelo;
    String color;
    int numPlazas;
}

/* PruebaCoche.java */
public class PruebaCoche {

   public static void main(String[] args) {

        //Creamos intancias de objetos
        Coche miCoche = new Coche();
        Coche tuCoche = new Coche();

    }
}
```

```java
/* Libro.java */
public class Libro {
    //Atributos
    String isbn;
    String titulo;
    String autor;
    int numPaginas;
}

/* PruebaLibro.java */
public class PruebaLibro {

   public static void main(String[] args) {

        //Creamos intancias de objetos
        Libro miLibro = new Libro();
        Libro librito = new Libro();
        Libro quijote = new Libro();
    }
}
```

3. Clases y Objetos

Clase Mesa

Atributos	Métodos
estado	comerEnMesa()
color	prepararMesa()
material	quitarMesa()
num_patas	estudiarEnMesa()
tamanyo	trabajarEnMesa()
superficie	subirseAMesa()

Objetos:

MesaCocina

Atributos: estado=”libre”, color=”gris”, material=”cristal”, numPatas=4, tamanyo=”grande”, superficie=”150x70cm”

Métodos: prepararMesa(), quitarMesa() y comerEnMesa())

MesaEstudio

Atributos: estado=”ocupada”, color=”marrón”, material=”madera”, numPatas=2, tamanyo=”mediano”, superficie=”70x40cm”

Métodos: trabajarEnMesa() y estudiarEnMesa())

3. Clases y Objetos

```
class NombreClase {

    //Atributos
    [public | private | protected ] tipoDato nombreVariable;

    //Constructores
    [public | private | protected ] constructor(parametros) {
        //Cuerpo de la función
    }

    //Métodos
    [public | private | protected ] tipoDevuelto nombreMetodoA(parametros) {
        //Cuerpo de la función
    }

    [public | private | protected ] tipoDevuelto nombreMetodoB(parametros) {
        this.nombreVariable=valor;
        this.nombreMetodoA();
    }
}
```

Nota: para hacer referencia a los atributos o métodos de la clase dentro de la clase se usa la palabra **this**

3. Clases y Objetos

```
public class Mesa {

     private String estado;
     private String color;
     private String material;
     private int numPatas;
     private String tamanyo;
     private String tamañoSuperficie;

    public void prepararMesa() {
          //Código necesario
     }
    public void quitarMesa() {
          //Código necesario
     }
    public void comerEnMesa() {
          //Código necesario
     }
    public void estudiarEnMesa() {
          //Código necesario
     }
    public void trabajarEnMesa() {
          //Código necesario
     }
}
```

Atributos de la clase Mesa

Métodos de la clase Mesa

4. Atributos y Métodos

Atributos

Las características que indican cómo es el objeto (**propiedades**)

Métodos

Las acciones que modelan cómo se comporta el objeto e indica lo qué puede hacer (**funciones** y **procedimientos**)

4. Atributos y Métodos

Los **atributos** (**propiedades** o **campos**) almacenan el valor de las propiedades del objeto.

- **Atributos de objeto** o instancia: Cada objeto toma un valor cuando se crea. Este atributo es accesible usando el nombre del objeto o con ***this*** desde el propio objeto.
- **Atributos de clase** (se declaran como **static**): Toma el mismo valor para todos los objetos de la clase (es como una variable global para todos los objetos). Es accesible sin necesidad de instanciar la clase.

Atributo de objeto: **mesaCocina.**estado = ”preparada”
Atributo de clase: **Mesa.**numMesas = Mesa.numMesas + 1

4. Atributos y Métodos

Los **métodos** son las **funciones** y **procedimientos** que puede ejecutar el objeto. Podemos distinguir entre:

- **Métodos de objeto** o instancia: Cada objeto llama a sus métodos con acceso a sus atributos. El método es accesible usando el nombre del objeto o mediante ***this*** desde dentro del propio objeto.
- **Métodos de clase** (se declaran como **static**): No se puede acceder a los atributos de instancia ni se puede usar con ***this***. Se accede sin necesidad de crear una instancia de la clase.

Método de objeto: **mesaCocina.**ponerMesa()
Método de clase: **Mesa.**totalMesas()

5. Constructores

- Para crear un objeto o una instancia de una clase, se debe reserva la memoria necesaria para guardar la información dando **valores iniciales a sus atributos**.
- La forma de hacerlo es con un método especial llamado **Constructor**. Por eso cuando creamos un objeto escribimos:

NomClase nombreObjeto = new Constructor(atributos)

- El constructor debe tener el mismo nombre que la clase.

```
public Coche (){}

public Libro (){}
```

5. Constructores

- En java, todas las clases tienen un **método constructor por defecto**, sin atributos.
- Como existe siempre suele omitirse cuando creamos la clase ya que normalmente se utiliza la **sobrecarga de métodos**.

```java
/* Coche.java */
public class Coche {
    //Atributos
    String matricula;
    String marca;
    String modelo;
    String color;
    int numPlazas;

    //Constructor por defecto
    public Coche() {}

}
```

```java
/* Libro.java */
public class Libro {
    //Atributos
    String isbn;
    String titulo;
    String autor;
    int numPaginas;

    //Constructor por defecto
    public Libro() {}

}
```

5. Constructores (Sobrecarga)

- La **sobrecarga** permite construir objetos de una clase utilizando diferentes valores iniciales de sus atributos.
- Para declarar un método sobrecargado se utiliza el mismo nombre pero debe tener diferentes atributos (tipo y/o número)

Ejemplo de sobrecarga:

```
public Mesa (){}

public Mesa (String color, int numPatas, String superficie){
   this.color = color;
   this.numPatas = numPatas;
   this.superficie = superficie;
}
```

```
Mesa mesa = new Mesa();
Mesa mesaCocina = new Mesa("Blanca", 4, "150x70cm");
Mesa mesaEstudio = new Mesa ("Marrón", 2, "70x40cm");
```

5. Constructores (Sobrecarga)

```
/* Coche.java */
public class Coche {
    //Atributos
    String matricula;
    String marca;
    String modelo;
    String color;
    int numPlazas;

    Public Coche() { }

    Public Coche(String matricula) {
        this.matricula = matricula;
    }

    Public Coche(String matricula, String marca, String modelo, String c, int plazas) {
        this.matricula = matricula;
        this.marca = marca;
        this.modelo = modelo;
        this.color = c;
        this.numPlazas = plazas;
    }

}
```

Encapsulamiento

Los objetos se comportan como una caja negra:

No se debe poder acceder a los atributos de instancia privados si no es a través de los métodos públicos disponibles.

Se pueden aplicar **modificadores** de acceso:

Modificador/Acceso	Clase	Paquete (package)	Subclase	Todos
public	SI	SI	SI	SI
protected	SI	SI	SI	NO
default	SI	SI	NO	NO
private	SI	NO	NO	NO

7. Getters y Setters

Los **atributos** de un objeto **deben ser privados**, no accesibles desde fuera de él.

Para leer o modificarlos se recurre a **métodos públicos.**

Estos métodos se llaman **Getters** (para leer) y **Setters** (para modificar)

Ejemplo: Para el atributo ***estado*** creamos ***getEstado*** y ***setEstado***.

```
class Mesa {
    //Atributos
    private String estado;
    //Métodos
    public String getEstado() {
        return this.estado;
    }
    public void setEstado(String nuevoEstado) {
        this.estado = nuevoEstado;
    }
}
```

Ejercicio

Crea una **clase** para los siguientes objetos.

Deberás **abstraer** los **atributos** necesarios así como indicar los **métodos**.

Crea un **constructor** y sobrecárgarlo para poder crear objetos sin atributos o con todos los atributos.

Crea **getters** y **setters** para los atributos privados.

